ENGINEERING STUDY EDITION / PHASES 0-7

SyncProof

The complete project guide

One coherent map of the protocol, code, failure evidence, measurements and decisions.

25 chapters in four parts
Eight dedicated phase chapters
Five end-to-end code paths
32 model answers and 10 adversarial follow-ups

Source and retained artifacts checked 05 November 2026.
No application test suite or benchmark rerun for this guide.
No implementation phase, commit or publication authorized.

Contents

Linked chapters and PDF bookmarks. Source tags such as [S2] jump to the final index. File targets there are repository-relative; some PDF readers restrict local file links. Keep this PDF under output/pdf or use the printed paths from the repository root.

I / Orientation and foundations 3

01 / A project you can explain . . . 3
02 / Why offline synchronization is difficult . . . 5
03 / Architecture and technology rationale . . . 6

II / The complete eight-phase story 7

04 / Phase 0 - Specify before building . . . 7
05 / Phase 1 - Make the server decisive . . . 9
06 / Phase 2 - Make local intent durable . . . 11
07 / Phase 3 - Recover uncertain HTTP outcomes . . . 13
08 / Phase 4 - Reconcile complete observations . . . 15
09 / Phase 5 - Let the user decide explicitly . . . 17
10 / Phase 6 - Challenge the whole protocol . . . 19
11 / Phase 7 - Measure and prepare for review . . . 21

III / Deep technical map 23

12 / Find the right code first . . . 23
13 / Complete data model and contracts . . . 24
14 / Five complete code paths . . . 26
15 / Correctness and concurrency reasoning . . . 28
16 / Client transactions and crash recovery . . . 30
17 / Reconciliation and conflict semantics . . . 31
18 / Failure-injection atlas . . . 32
19 / Performance and complexity . . . 34
20 / Security, operation and future alternatives . . . 36

IV / Defend and extend it independently 37

21 / Run and reproduce safely . . . 37
22 / Interview questions and adversarial follow-ups . . . 39
23 / A ten-session independent learning plan . . . 43
24 / Resume and demonstration guidance . . . 45
25 / Final audit and twelve things to remember . . . 47
Source and evidence index . . . 49

I / Orientation and foundations

Start with ownership and uncertainty. The code is small enough to inspect, but its safety depends on keeping several kinds of knowledge separate.

01 / A project you can explain

Purpose, evidence and a reading map

SyncProof is a local field-inspection application built to study what happens when a browser and a database disagree about whether a request finished. Inspectors edit one record type with title, status and notes. They save locally, explicitly freeze an immutable operation, submit it and reconcile with authoritative PostgreSQL. Another inspector can change the same record; the loser keeps a visible proposal rather than silently overwriting the winner. The field domain is deliberately ordinary so the synchronization reasoning stays visible. **[D1] [D2]**

The intended reader knows programming and basic SQL but wants to defend backend and recovery decisions in an interview. This guide follows all eight phases, then reconnects their mechanisms into the final implementation. It is not a new implementation phase. It does not establish independent owner mastery or claim that AI-assisted implementation was independently authored by the owner. **[P0] [P7]**

What exists and what the evidence says

The final repository implements manual foreground synchronization, whole-record revision checks, durable original receipts, preserved local drafts, complete snapshots, tombstones and explicit conflict resolution. It deliberately omits authentication, automatic merging, restoration, cursors, background synchronization, multi-tab coordination and deployment. The retained Phase 7 release record reports 123 named cases passing, two complete ordered demo repetitions and 4,010 benchmark observations. This documentation session inspected those artifacts and source hashes; it did not rerun the application tests or benchmarks. Those are different evidence claims. **[D11] [E1] [E2]**

Phase	Capability added	Acceptance / evidence
0	Contract and failure model	14 documentation gates; no app tests
1	Authoritative mutation server	16 gates; 45 backend cases
2	Durable local authoring	18 gates; 13 local, 4 browser
3	Unknown-outcome push/replay	20 gates; 9 push, 15 browser total
4	Complete pull/reconciliation	22 gates; 4 snapshot, 22 browser total
5	Manual resolution/deletion	24 gates; 5 resolution, 31 browser total
6	Deterministic full fault story	22 gates; 123 named cases, 44 browser
7	Measurements and owner-review readiness	20 gates; same 123 cases, 4,010 timings

These phase counts are milestone snapshots, not numbers to add together: each later suite includes earlier regressions. The final 126 main command executions include three repeated process checks; the fresh-copy 18 contract cases repeat existing cases. Timed samples are not additional correctness test cases. Original reports and evidence remain the authoritative historical record. **[P0] [P1] [P2] [P3] [P4] [P5] [P6] [P7]**

How to study this book

- First pass: chapters 1-3 and the eight phase chapters. Explain what each phase made possible and what it still could not claim.

- Second pass: contracts, five code paths and the invariant map. Keep the repository open at the referenced symbols.

- Practical pass: runbook, failure atlas and performance chapter. Reproduce a bounded scenario before changing it.

- Interview pass: answer the grouped questions without reading the outlines, then use the adversarial follow-ups.

Small glossary

Term	Meaning in this repository
Transaction	A group of database operations that commits together or rolls back together; SQL and IndexedDB have separate transactions.
Atomicity	The relevant state transition becomes durable as a whole, not an arbitrary subset.
Revision	A server-owned version of one record, represented as a positive decimal BIGINT string.
Optimistic concurrency	Submit the version you edited; reject if current authoritative state has advanced.
Idempotency	Replaying the same issued ID and immutable content returns its original outcome without another effect.
Canonicalization	Turn validated content into one defined byte representation for identity comparison.
Receipt	Durable original terminal decision for one mutation, not necessarily current record state.
Tombstone	A retained, versioned deleted row that reserves identity and tells old clients about deletion.
Invariant	A stated property that transitions are intended to preserve, under explicit assumptions.
Unknown outcome	A transmitted or possibly transmitted request without a locally committed terminal receipt.

Source and evidence: **[D1] [D2] [D3] [D6] [E1]**

02 / Why offline synchronization is difficult

The same base can produce two valid intentions

Imagine two inspectors both reading revision 4. A changes the notes and B changes status while offline. Each edit is reasonable relative to what its author saw. If the server accepts both as unconditional replacement records, the last arrival can erase the earlier one. Whole-record optimistic concurrency instead accepts one base 4 mutation as revision 5 and rejects the other as stale. That rejection is not a lost proposal: the browser preserves the base, intent and decision for a deliberate next step. This is an illustrative schedule; observed base 4 examples are in the resolution tests. **[S2] [C2] [T11]**

Now lose A's response after its commit. A sees silence, while the server holds revision 5 and a receipt. If B later commits 6, comparing A's intended values with 6 cannot tell whether A committed 5 or never reached the server. This is why a timeout is not a conflict decision. Retry the same issued identity, obtain its original outcome and only then treat a new snapshot as the latest observation. A historical success 5 and current 6 can both be true. **[C3] [E4] [E5]**

Acknowledged means a valid server outcome was durably installed locally. Queued means an immutable request exists locally but may never have been attempted. Unknown means no local terminal evidence settles a possibly committed request. Terminal conflict means the server's decision for that request is final, while the user's proposal may still need resolution. These are not interchangeable UI adjectives. In particular, terminal does not mean the user's work has been accepted. **[C2] [C3] [C5]**

What the custom protocol chooses to teach

SyncProof uses a server-owned linear revision per inspection and a durable receipt per operation. This makes identities and transaction boundaries inspectable. It is not a claim to improve on established products. The repo's sourced comparison describes RxDB checkpoint-based replication and conflict-state exchange, Electric's Postgres read-path Shapes with application write patterns, and PouchDB/CouchDB revision-tree replication and tombstones. Those systems have different purposes and operational capabilities. SyncProof reimplements a small learning subset, not those platforms. **[D1] [D7]**

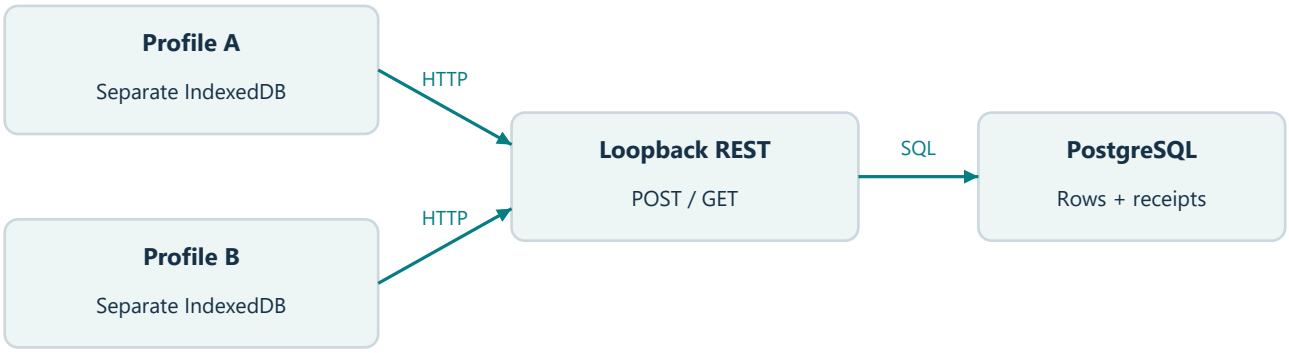
Production alternatives would also require attention to authorization, operational recovery, compatible upgrades, retention, observability, larger-scale replication and supported browser behavior. SyncProof deliberately avoids making those promises. Choosing a simpler full snapshot removes cursor bookkeeping but increases repeated transfer; choosing whole-record revision checks avoids automatic merge policy but causes conflicts even for disjoint field changes. Both are explicit tradeoffs. **[D2] [D5] [D9]**

Sourced tool	Documented purpose / contrast
RxDB **[X1]**	Checkpoint pull, assumed-master/new-fork push and conflict state exchange, plus live observation; SyncProof uses its own foreground receipts/full snapshots.
Electric **[X2]**	Postgres read-path HTTP Shapes, with application writes discussed separately; not identical to this custom mutation API.
PouchDB/CouchDB **[X3]**	Revision-tree replication with deterministic winner/conflict branches and deletion markers; SyncProof keeps one authoritative linear revision and local rejected proposals.

03 / Architecture and technology rationale

One authority, independent stores and bounded tools

React renders forms and durable-status observations. TypeScript describes validated shapes but does not replace runtime parsing. Dexie wraps the browser's IndexedDB transactions. Express supplies the narrow HTTP boundary; PostgreSQL supplies constraints, locks and commits. The inspection-specific code connects those mechanisms into immutable identity, receipts, recovery and explicit decisions. The system has one authoritative database, not peer databases trying to elect a winner. **[U7] [S3] [C5] [D1]**



Current server rows, preserved intent and historical receipts are different facts.

Dependency / tool	Why it exists	What it does not supply
React / react-dom	Minimal form, selection, status and conflict UI	No durability or distributed commit
TypeScript	Static shape checking for server, client and tests	No runtime trust in HTTP/IndexedDB data
Dexie / IndexedDB	Indexed local stores and atomic multi-store transitions	No server replication or physical durability guarantee
Express / pg	HTTP routing and PostgreSQL connections	No business idempotency automatically
PostgreSQL	Unique claims, row locks, expected-revision writes, receipts	No browser acknowledgment transaction
Playwright / node:test	Real browser/process and logic/integration checks	No proof over every failure schedule
fake-indexeddb	Fast local transaction/state simulations	Not real browser persistence evidence
Vite	Loopback frontend and /api development proxy	Not production hosting or offline shell caching

Kafka, Redis, microservices and a general replication framework were excluded because the educational problem needs neither a distributed event bus nor another storage authority. More infrastructure would add failure boundaries before the basic two-store protocol was understood. The code instead keeps one server write endpoint, one complete read endpoint and a foreground client controller. This is a scope decision, not a universal systems-design recommendation. **[D1] [D4]**

The browser calls /api/mutations or /api/inspections on Vite at 127.0.0.1:5174. The proxy strips /api and forwards to 127.0.0.1:3000. Server startup validates a loopback HTTP Origin and binds loopback. Origin filtering is not authentication: other local processes can still submit requests. The helper PostgreSQL cluster is separate on 55432 and uses local trust for synthetic data. **[U8] [S5] [U1]**

Source and evidence: **[S1] [S2] [S3] [S4] [S5] [C1] [C2] [C3] [C4] [C5] [C6]**

II / The complete eight-phase story

Read each phase as a response to the risk left open by its predecessor. A completed gate authorizes review, not automatic expansion.

04 / Phase 0 - Specify before building

Scope, assumptions and an evidence vocabulary

Problem and sequence

Before there was a server or a browser queue, the project needed a precise meaning for success, uncertainty, deletion and eventual recovery. Without that contract, later green tests could prove only whichever behavior happened to be implemented. Phase 0 makes the intended obligations reviewable before code exists. **[P0]**

Starting state, objective and boundary

The workspace was a documentation baseline. The objective was a one-inspection MVP and eight gated phases, not an application, benchmark or deployment. Its useful outputs are charter, protocol, invariant registry, failure specifications, evidence register, decisions and roadmap. All runtime evidence was explicitly NOT RUN. **[D1] [D2] [D3] [D5] [D6]**

Implemented mechanisms and source

The protocol assigns server-owned revisions, immutable operation IDs and lifetime receipts/tombstones. The invariant registry gives stable names such as INV-IDEMP-01 and INV-UNKNOWN-01; F1-F10 describe initial state, injection point, client/server expectations and recovery. The evidence register separates desired claims from executed support. These are documentation mechanisms, not working runtime enforcement. **[D2] [D3] [D6]**

Choices and rejected alternatives

Complete snapshots, one active controller per profile and manual whole-record resolution limit the state space. Automatic merge, cursor retention, restoration and competing tabs were postponed. The choice favors an understandable protocol at the cost of bandwidth and human conflicts. It does not claim the simpler design scales indefinitely. **[D4] [D5]**

Execution walkthrough

Take the planned lost-ack schedule: freeze m from revision 4, let the server commit 5, lose the reply, let another writer commit 6, then restart. Write down what each side knows before choosing behavior. The contract says original m must be recovered through its receipt; neither timeout nor snapshot 6 may invent a terminal result. This was a Phase 0 specification, later exercised by real code. **[D2] [P0]**

Evidenced edge cases and fixes

There was no implementation bug to claim fixed because no application existed. The important work was making assumptions explicit: retained storage/database incarnation, tombstones, atomic acknowledgment, real process tests distinct from exceptions, and no cold-load offline shell promise. Later reports refine details rather than converting Phase 0 planning into retroactive execution evidence. **[P0]** **[D4]**

Acceptance and executed evidence

All 14 documentation acceptance items passed structural/manual checks. No application tests, database restart, browser lifecycle or benchmark ran. Treat those 14 PASS labels as specification completeness only. The original report's source/tool comparison is a documented comparison, not an installed-product evaluation. **[P0]**

Complexity and limits

O(N) complete transfer and lifetime growth were design expectations, not measurements. Conditional convergence requires finite/quiescent writes, working retained storage, repeated reachable opportunities and eventual user decisions. No phase can remove those conditions by changing a label to synced. **[D2]** **[D5]**

Explain it yourself; independent exercise

Draw the planned 4/5/6 lost-ack schedule without code. Hint: keep current row, receipt and local proposal on separate lines. Success means explaining why the same visible final row could follow either a committed or an unadmitted request, and why only original identity settles it. This exercise is not completed for the owner. **[P0]**

Deliberately postponed

Phase 1 had to turn the write contract into actual SQL/HTTP behavior. The specification did not authorize a UI, transport retries or full F1-F10 certification. Each later implementation phase would add executed evidence at its own layer. **[D5]**

05 / Phase 1 - Make the server decisive

Atomic effects, original receipts and real SQL races

Problem and sequence

The contract needed an authority that could decide one request once even if delivery repeats or processes fail. A memory map cannot arbitrate across a restart, and a check outside the write transaction can become stale. PostgreSQL must own both the revision transition and proof of processing. **[P1] [S2]**

Starting state, objective and boundary

Starting from documents, Phase 1 added a Node/TypeScript REST server, SQL migration and real integration/process tests. Only POST /mutations was implemented then; the later GET endpoint and browser were deliberately absent. A deferred endpoint's historical 404 in that report is not a contradiction with today's Phase 4 implementation. **[P1] [S3]**

Implemented mechanisms and source

parseMutation validates a strict envelope and returns canonical bytes/fingerprint. processMutation claims processed_mutations before record access, handles replay/collision or adjudicates create/update/delete, fills the terminal response and commits. inspections has per-record BIGINT revision/deleted state. A deferred constraint rejects incomplete receipt claims at commit. **[S1] [S2] [S6]**

Choices and rejected alternatives

The code rejects memory-only deduplication, separate receipt/effect commits and last-write-wins. A unique claim handles the SAME ID; a row lock plus revision predicate handles DIFFERENT IDs on the same record. They solve different races and neither replaces the other. No ORM or extra queue service was needed. **[P1] [S2]**

Execution walkthrough

A valid create claims its ID, inserts inspection 1, stores its committed outcome and commits both. An exact duplicate waits if necessary, reads the stored receipt in a later READ COMMITTED statement and returns replayed=true. Changed canonical content under that ID returns collision without effect. A new stale update ID receives a durable conflict rather than a transient transport error. **[S2] [T2]**

Evidenced edge cases and fixes

The report records a real contract clarification: PostgreSQL text/jsonb cannot store NUL, so validation now rejects U+0000 before admission instead of producing an indefinite internal failure. It also records tightening an unmatched-quote charset regex and tests. Dependency/process EPERM and database startup capture issues were environment failures, not hidden passing cases. **[P1] [S1] [S3]**

Acceptance and executed evidence

All 16 gates passed. Final execution was 18 contract tests plus 27 real SQL/process cases:45, with no final skips. Actual Node termination/restart ran after provisional write before commit and after commit before response. Simulated throws were recorded separately. Current source still uses the same core algorithm, although later phases add reads and client behavior. **[P1] [E17] [T3]**

Complexity and limits

Canonicalization/hash costs O(B). Indexed identity/record search has B-tree-like size/cache effects plus lock and commit cost, not guaranteed constant time. Receipts retain payload history forever in this MVP. Application-process restart tests retain PostgreSQL; they do not restart the database, OS or disk. **[P1] [S4]**

Explain it yourself; independent exercise

Annotate BEGIN, claim, row lock, conditional update, receipt completion and COMMIT in processMutation. Hint: draw a concurrent identical contender and a different-ID contender separately. Success means predicting commit/rollback/replay for each checkpoint and naming the actual process test that supports your claim. **[S2] [T3]**

Deliberately postponed

The server could preserve decisions, but no browser yet had durable drafts or knew how to remember an issued request. Phase 2 therefore had to solve local identity and atomic freezing before any HTTP retry loop was safe. **[P2]**

06 / Phase 2 - Make local intent durable

Editable work is not an issued operation

Problem and sequence

A reliable server is insufficient if a browser forgets the request identity between clicks or restarts. Before sending, local authoring must distinguish unsaved typing, a committed editable draft and a frozen operation that can no longer change. Phase 2 establishes that boundary without pretending local storage means server acceptance. **[P2]**

Starting state, objective and boundary

The Phase 1 server already exists. Phase 2 adds a minimal React/TypeScript client, browser contract copy and Dexie stores, plus local/browser tests. It does not implement real transport or conflict resolution. A storage-only synthetic receipt seam tests future acknowledgment transaction foundations and is explicitly labeled synthetic. **[P2] [C1] [C2]**

Implemented mechanisms and source

LocalStore.initialize commits a stable random client identity. work stores drafts/locks, mutations stores immutable issued requests plus mutable processing metadata, cleanRecords stores authoritative observations, and metadata stores profile/snapshot information. saveDraft increments a local version while retaining the original server base. freeze prepares a UUID/canonical/hash and atomically creates the queue entry and lock. **[C2]**

Choices and rejected alternatives

Hashing finishes before the read-write transaction because unrelated asynchronous waits can let IndexedDB become inactive. The transaction rechecks draft version and active slot after preparation, rather than trusting an earlier read. Separate clean/work stores avoid replacing a user's proposal when server state later refreshes. There is no arbitrary local dependency graph. **[C2] [P2]**

Execution walkthrough

Type a new inspection, Save draft and wait for completion. The saved work has a null server base and local version 1. Queue reads it, prepares immutable content and then commits mutation+locked work together. Reload or graceful process reopen recovers the same identity/envelope. If the staged write fails, neither a phantom queue nor a stranded lock may remain. **[T4] [T8]**

Evidenced edge cases and fixes

The report records a saved-null-base fallback bug: treating a stored null like missing information could silently rebase a local create when a clean fixture appeared. It was corrected and regression-tested. Synthetic outcome equality also became independent of JSON member order. Initial typing/CSS/JSX and Windows spawn issues were distinct from the final successful persistence checks. **[P2] [T4]**

Acceptance and executed evidence

All 18 gates passed. Final 13 local cases use fake-indexeddb and do not prove real disk behavior; four Chromium cases provide actual browser workflows and a whole persistent-browser close/relaunch with different PIDs. The 45 backend cases reran. Fresh-profile isolation is different evidence from two tabs sharing one store. **[P2] [E18] [E19]**

Complexity and limits

Preparation/copy/hash scales with payload bytes; indexed local reads/writes and rendering depend on retained store sizes. There is no pruning. Completed storage survives the tested graceful lifecycle, not eviction, profile deletion, physical failure or uncommitted typing. The page shell must remain reachable after restart. **[P2]**

Explain it yourself; independent exercise

Read freeze and draw a failure after mutation insertion but before work lock. Hint: inspect both stores after transaction abort, not only a disabled button. Success means explaining why no issued ID is visible without its lock, and why hashing itself cannot sit inside the transaction. **[C2] [T4] [T8]**

Deliberately postponed

A durable queue still does not know whether a remote attempt committed. Phase 3 adds real HTTP, a pre-send unknown marker and validated atomic acknowledgment while reusing the same frozen bytes. **[P3]**

07 / Phase 3 - Recover uncertain HTTP outcomes

An issued identity outlives an individual attempt

Problem and sequence

Once requests can leave the browser, a timeout can hide a committed mutation. Retrying with a replacement ID could apply the operation again. Phase 3 makes uncertainty durable before transmission and requires original-ID replay rather than guessing from HTTP availability. **[P3] [C3]**

Starting state, objective and boundary

The server and local queue work independently. Phase 3 joins them with explicit Send pending / Retry, bounded retry and real receipt installation. It remains push-only: no complete authoritative pull or claim of multi-client clean reconciliation yet. Editable drafts are never automatically frozen by a send. **[P3] [C5]**

Implemented mechanisms and source

deliver sends the stored canonical body, times request/body handling and classifies terminal, unknown or blocked responses. PushController shares one active promise, processes unknown before queued and rechecks stored integrity each attempt. markAttempt commits outcome_unknown before network I/O. applyReceipt validates and atomically records outcome, clean handling and work transition. **[C3] [C2]**

Choices and rejected alternatives

Five attempts use bounded exponential jitter and a 10-second timeout. A Retry-After beyond 30 seconds persists notBefore instead of holding the UI indefinitely. Known collision/admission diagnostics preserve identity; a current rejected attempt cannot erase uncertainty about earlier attempts. Network calls stay outside Dexie transactions. No online-status hint is treated as proof of outcome. **[C3] [T5]**

Execution walkthrough

The browser freezes m, durably marks it unknown, POSTs and loses the response after SQL commit. The test checks the actual receipt before aborting delivery. Local storage retains exact m and its lock. A later same-ID retry obtains replayed=true and the original terminal outcome, then one local acknowledgment transaction releases successful work. **[T9] [P3]**

Evidenced edge cases and fixes

Receipt validation was strengthened beyond the earlier synthetic seam, including impossible create-conflict cases. The report records preserving long Retry-After even when an error body is unreadable, correcting an optional-header test annotation and making sending status distinct from saving. A native receipt-put failure after real commit leaves unknown recoverable; it is a deliberate fault test, not an incident of actual disk exhaustion. **[P3] [T5] [T9]**

Acceptance and executed evidence

All 20 gates passed. Nine controller/transport cases and 15 browser cases (four prior plus 11 new) passed with local 13/backend 45 regressions. Real committed-response loss, identity collision, native acknowledgment abort and retained-profile unknown/acknowledged restart are evidenced. Full pull, transport reorder across clients and interrupted manual resolution were still future work then. **[P3] [E20]**

Complexity and limits

Eligible work is indexed/materialized and sent sequentially, with bounded attempts per cycle; each attempt also validates/hashes bytes. A retry budget bounds one opportunity, not eventual delivery. Permanent unreachability or blocked integrity errors can leave work unknown indefinitely without authorizing discard or a new identity. **[C3] [P3]**

Explain it yourself; independent exercise

Inspect markAttempt, deliver and applyReceipt. Hint: interrupt each boundary before/after SQL commit and before local receipt commit. Success means predicting durable states and explaining why both a transport error and a local storage error can require the same ID, although they happened on opposite sides of HTTP. **[C2] [C3]**

Deliberately postponed

Push acknowledgment is not a view of all server rows. Phase 4 must implement complete consistent reads and safe local installation without rewriting pending intent or confusing historical receipts with the latest state. **[P4]**

08 / Phase 4 - Reconcile complete observations

A snapshot is coherent, not permanently current

Problem and sequence

Clients now know outcomes of their own sends but still lack a complete server view. The next risk is read-side loss: installing a partial, older or missing-row response could regress clean data or silently erase a draft. Phase 4 makes snapshot completeness and installation explicit. **[P4]**

Starting state, objective and boundary

Starting with push/retry, this phase adds GET /inspections and foreground recover-pull-push-pull. It keeps full snapshots rather than inventing cursors. Resolution UI remains absent at this milestone; conflicts are preserved, not silently solved. **[P4] [S3] [C4]**

Implemented mechanisms and source

The endpoint uses one SELECT ordered by id, including tombstones, with revision::text; count derives from those rows. validateSnapshot checks the whole typed response before writing. applySnapshot atomically updates cleanRecords/lastSnapshot after monotone, equal-state, missing-ID and tombstone checks. It never writes work or mutation history. **[S3] [C4] [C2]**

Choices and rejected alternatives

Separate count/page queries could observe different commits; a single statement avoids that mixed-view contract. Absence of a known authoritative ID is rejected rather than interpreted as deletion because tombstones must remain. The running-promise guard also prevents overlapping Send from falsely reporting a completed pull through snapshotApplied. No global version is inferred from UUID order. **[P4] [C3]**

Execution walkthrough

A and B pull the same record. B saves an unsent draft; A changes the server. B's pull advances only cleanRecords, retaining original draft/base. Freezing that old-base work can produce a genuine stale conflict. If a delayed older snapshot arrives after a newer install, the entire response is rejected; metadata must not claim a fresh complete observation for mixed local state. **[T10] [T6]**

Evidenced edge cases and fixes

The old deferred GET 404 expectation intentionally changed to 200. A misleading remote-base warning for a local-only draft was corrected, and shared foreground status was tightened. One continued verification found PostgreSQL stopped: seven setup failures were retained, the owned cluster restarted and cases rerun. That environment failure was not relabeled a passing body. **[P4]**

Acceptance and executed evidence

All 22 gates passed. Four snapshot-local and seven new snapshot-browser cases joined a 22-browser total, with previous 45/13/9 backend/local/push regressions. Three isolated clean caches converged under quiescence while one draft remained. Virtual 21-day/tombstone observations are simulated duration, not weeks of physical retention. **[P4]** **[E21]**

Complexity and limits

Each pull materializes/transfers O(NB), validates O(N) records and performs known-row checks/indexed writes. Two pulls add repeated transfer. No pruning or production-scale guarantee exists, and even a coherent successful read can be stale immediately after its SQL statement. **[D5]** **[P4]**

Explain it yourself; independent exercise

Construct a purported complete response missing a known tombstone. Hint: compare clean/work/mutations/metadata before and after rejection. Success means explaining why local-only absent creates remain safe while missing previously authoritative IDs block the whole install. **[C2]** **[T6]** **[T10]**

Deliberately postponed

Preserved conflicts need human decisions. Phase 5 adds explicit reviewed keep/reapply/manual-combine without changing server identity, revision checks or unknown recovery. **[P5]**

09 / Phase 5 - Let the user decide explicitly

New resolution attempts, never magic overwrites

Problem and sequence

A durable stale proposal is safe but not yet usable. The user needs to compare what was originally read, what was proposed, what the server decided and what is currently known. Phase 5 makes that lifecycle explicit while preserving original terminal evidence. **[P5]** **[C6]**

Starting state, objective and boundary

The read/write engine already works. This phase adds minimal decision UI and local resolution states, confirmed deletion and second-conflict handling. It does not add automatic merging, restoration, queue chains, server drafts or a new protocol. **[P5]**

Implemented mechanisms and source

ConflictPanel shows base, rejected local operation, historical receipt and latest known remote. beginResolution stores an editable candidate with resolutionOf/intent and reviewed base. freeze issues a new immutable ID with resolves provenance. keepServer records confirmed local decision metadata and removes active unresolved work without POST; cancelResolution restores its predecessor proposal. **[C6]** **[C2]**

Choices and rejected alternatives

Review latest uses recovery+pull, not queued push. Reapply selects rejected values; manual combine starts from remote values for explicit editing. Tombstones prohibit implicit restore; a create collision cannot become an upsert. A stale delete against live state requires another destructive confirmation/new ID; an already-deleted row requires no automatic extra delete. **[C2]** **[C3]** **[C6]**

Execution walkthrough

In a real base 4 fixture, A wins 5 and B conflicts. B prepares a new base 5 resolution; another writer reaches 6 first, so that resolution conflicts too. A further manual candidate/new ID/base 6 commits 7; both clients reconcile. A separate F9 branch commits 6, loses the reply, restarts Chromium and replays the same resolution while clean state later reaches 7. They are separate tests, not one invented combined trace. **[T11]** **[E5]**

Evidenced edge cases and fixes

The first browser attempt selected the older parent after a second conflict and disabled relevant actions. Selection now uses current activeMutationId, while a draft points to its predecessor; keyed panel state resets review for a new conflict. Another initial failure was Vite HMR from editing source during a run. Both failures were retained; final runs avoided source edits mid-suite. **[P5]**

Acceptance and executed evidence

All 24 gates passed. Five local resolution cases and nine real resolution-browser cases produce 31 browser cases and 107 combined named cases at that milestone. Native freeze/ack aborts, repeated clicks, actual graceful process restart, update/delete/tombstone cases and simulated weeks passed. The full ordered demo was still unrun then. **[P5]** **[E22]**

Complexity and limits

Candidate validation/hash is O(B), point storage operations are indexed and history remains retained. Explicit review costs a pull but cannot lock out a later writer. Choices do not run while the same record has a pending/unknown resolution. General conflict graphs and cross-tab arbitration remain excluded. **[C2]** **[P5]**

Explain it yourself; independent exercise

Reproduce the second-conflict branch. Hint: keep original m, rejected resolution r and final new r2 distinct, with base 4/5/6 and each terminal receipt. Success means explaining why neither changing r's payload nor labeling receipt 5 as current 6 is valid. **[T11]**

Deliberately postponed

Targeted examples existed, but a reproducible whole failure atlas and complete ordered interview story still needed consolidation. That is Phase 6 testing work, not another feature wave. **[P6]**

10 / Phase 6 - Challenge the whole protocol

Deterministic schedules and one ordered story

Problem and sequence

Green isolated cases were not yet a reproducible account of the whole workflow. Phase 6 had to force missing delivery/competition schedules and connect client storage, actual SQL outcome and real process lifecycle. Its output is a bounded audit, not new business features or a universal proof. **[P6]**

Starting state, objective and boundary

Phase 5 supplied 107 named cases and targeted F9. Strong tests were reused. Gaps included before-HTTP-admission reorder in both orders, coordinated independent-browser competition, per-scenario evidence and one ordered eight-step demo. No protocol rewrite was authorized. **[P6]** **[E16]**

Implemented mechanisms and source

audit-harness creates generated schemas/servers and separate browser stores or persistent profiles. Promise gates hold response/admission; existing mutation hooks hold SQL checkpoints; parent IPC supervises actual Node children. JSON records scenario/run IDs, durable observations and caveats. Native IDB interception exists only in tests. **[H2]** **[H3]** **[T12]** **[T3]**

Choices and rejected alternatives

No arbitrary sleep selects a winner. F2 observes a PostgreSQL waiter blocked; F3 holds the first-sent request before admission; F8 holds both before record locking and releases in chosen orders. Response loss was tightened to hold delivery until reload destroys the controller, removing a reload-versus-backoff timing window. Normal runtime exposes no crash endpoint. **[H2]** **[P6]**

Execution walkthrough

The demo starts two persistent clients at 1: A commits 2, B conflicts, B resolves with new ID/base 2 to 3. A separate create commits 1 but loses delivery, then exact replay acknowledges it. A different offline create survives whole Chromium restart. A deliberate delete produces tombstone 2; final reconciliation has three rows, seven receipts and no active work on both clients. Two clean-fixture runs assert all eight ordered steps. **[T13]** **[E9]** **[E10]**

Evidenced edge cases and fixes

A new gate helper first failed TypeScript checking because of generic release inference; explicit void corrected it. No production defect needed a fix. The response barrier strengthened scheduling. Simulated exceptions, forced Node exit, graceful browser close and clock advance remain separately labeled modalities. **[P6]**

Acceptance and executed evidence

All 22 gates passed. Eleven audit-browser cases and two ordered demo repetitions joined 31 earlier cases for 44 browser cases. Three new database/safety cases joined 45 existing backend cases; with 13 local,9 push,4 snapshot and 5 resolution there are 123 named cases. Final commands executed 126 because three process checks repeated. Matching-run artifacts and logs support the count, not accumulated earlier passes. **[P6]** **[E16]**

Complexity and limits

Fixtures/process startup and copying trace state add bounded harness cost, not a general chaos platform. F6 forces Node exit while retaining PostgreSQL; its local store assertions use fake-indexeddb. F5/F9 separately exercise actual graceful Chromium lifecycle. Unsupported modalities are not inferred from their success. **[E6]** **[E7]** **[E8]**

Explain it yourself; independent exercise

Choose an F-row and design a gate proving the fault occurred. Hint: pair SQL receipt queries with durable unknown assertions. Success means predicting both sides before recovery and identifying exactly which process was real versus simulated. A thrown exception is not a restart. **[D3]** **[H2]**

Deliberately postponed

Measurements, educational synthesis and publication review belonged to Phase 7. Passing fault schedules did not authorize deployment, benchmark claims or license choice. **[P7]**

11 / Phase 7 - Measure and prepare for review

Local evidence, not automatic publication

Problem and sequence

The protocol needed reproducible setup, measured costs and explanations another engineer could inspect. Phase 7 audited actual Phase 6 evidence, then reran checks before measuring. It separated technical completion from public-sharing decisions and owner understanding. **[P7]**

Starting state, objective and boundary

The 123-case audit existed. Phase 7 added tools/docs, not product features. No application source, contract, migration, dependency or package-script change was needed. Four existing test files only route current outputs separately to preserve older evidence. **[P7] [E1]**

Implemented mechanisms and source

bench.mjs invokes real HTTP/SQL/browser modules in seeded schemas. reproduce.mjs checks a disposable independent source copy. demo-clients opens retained manual profiles and has a disposable headless check. release-audit inventories candidates/dependencies; verify-release checks artifacts/statistics/links. README, handbook and reports expose actual observations. **[U2] [U3] [U4] [U5] [U6]**

Choices and rejected alternatives

Warm HTTP, browser installation and foreground times use different clocks. Statistics use raw samples; p95 is omitted below 100 observations. Same-record races are separate from independent writes. No framework or speculative optimization was added merely because snapshots grow with N. Licensing remained an owner choice. **[D8] [D9]**

Execution walkthrough

Checks pass before measurement. Two schema/context repetitions cover N=100/500/1000 with 10% tombstones, Q=10/50/100 and writers 1/2/10/20. The run retains 4,010 observations/32 groups plus SQL plans/storage sizes. Fresh-copy online lockfile install then builds/tests/smokes independently. The profile check closes/removes only its disposable profiles. **[E2] [E14] [P7]**

Evidenced edge cases and fixes

Initial smoke failed before samples: static Playwright import chose a default cache before project path assignment. Dynamic import after environment setup fixed discovery. Offline fresh installation lacked a cached tarball; network-enabled retry passed. Permission and authoring-command limits were setup issues, not product defects. Failed attempts remain retained. **[P7]**

Acceptance and executed evidence

All 20 gates passed for local deliverables. The 123 named cases, both demos, builds/strict checks, fresh-copy smoke and launcher check passed. Registry audit reported zero known advisories, not absence-of-vulnerabilities proof. Phase 7 recorded 799 links/anchors checked. This guide session only inspected those prior results; it did not execute those tests again. **[E1] [E11] [E15]**

Complexity and limits

Accepted/replay/stale HTTP medians were 4.995/3.401/4.156 ms (n=200 each). A 1,000-row body was 455,017 bytes; combined browser install median 286.350 ms (n=100). Full N=1000/Q=100 median 2193.800 ms has n=10, so no p95. N/Q co-vary and throughput varies across repetitions; no causal scaling or SLA follows. **[E2] [D8]**

Explain it yourself; independent exercise

Recompute one median from raw samples. Hint: never mix Node body timing, browser applySnapshot and full synchronize. Success means naming warmup, n, included/excluded work and a confounder, not claiming one warm machine proves deployment capacity. **[U2] [E2]**

Deliberately postponed

There is no automatic Phase 8. Public push is a no-go pending license/private-path review and explicit approval. Owner exercises and peer review are also not established by a generated book. **[D9] [P7]**

III / Deep technical map

Use these chapters to trace failures, review changes and explain why similar-looking requests may require different identities.

12 / Find the right code first

Repository responsibilities and a debugging matrix

src owns HTTP, contracts and SQL; migrations owns durable rules. client/lib owns validation/storage/push/pull, while main.tsx/conflict.tsx render deliberate actions. tests separates simulations, SQL and actual lifecycle. scripts owns setup, measurements and review tools; docs records design/history/observations. **[S1] [C2] [U7] [D6]**

When debugging...	First symbols	Next evidence
A duplicate increments twice	processMutation claim/replay; receipt PK	T2/T3 and F2
Blocked/uncertain send	deliver; run; noteFailure	T5/T9, F1/collision
Lost saved proposal	saveDraft; freeze; applyReceipt; applySnapshot	T4/T6, native aborts
Clean revision rollback	applyReceipt/applySnapshot checks	F1/F9, delayed snapshot
Wrong conflict actionable	App activeMutationId/resolutionOf	T11 second-conflict case
Deleted ID returns live	adjudicate; queueDelete; applySnapshot	F7/F10, tombstones
Hung or leaking test	audit gates/finally; process launcher	P6 and cleanup checks
Misleading performance	bench clocks/config/raw groups	E2; no guessed optimization

Server data types live in src/contracts.ts and browser counterparts in client/lib/contract.ts. Work/Mutation storage metadata is in storage.ts. TypeScript unions are not runtime validation: parseMutation, validateReceipt and validateSnapshot defend actual inputs. A cast of HTTP JSON is not proof. **[S1] [C1] [C2] [C4]**

package.json defines build/test entry points. npm test builds then runs contracts and actual SQL/process suites. Playwright uses one worker/zero automatic retries and owns loopback ports. Browser fixtures import compiled server modules so migration-relative paths resolve. Hooks are programmatically injected or guarded by test mode/IPC, not normal UI actions. **[U7] [U9] [H2] [S5]**

13 / Complete data model and contracts

Truth, intent, history and presentation

Object / identifier	Owner and role	Important distinction
Inspection id	Client-chosen identity, server committed row	Not mutation identity or revision
mutationId	Globally unique immutable operation	Changed content/base needs new ID
clientId	Persistent profile tracing identity	Not authentication or tenant scope
baseRevision	Version expected by update/delete	Original draft does not silently rebase
cleanRecords	Latest observation known locally	May exceed historical receipt revision
commonBase / work.payload	Observed base and local intended values	Preserved across pull/conflict
canonical / fingerprint	Defined bytes and SHA-256	Server compares both
receipt / resolves / decision	Outcome, predecessor link, local choice	Metadata does not rewrite outcome
deleted	Retained authoritative tombstone	Absence is not equivalent deletion

An envelope has exactly protocolVersion, mutationId, clientId, recordId, operation, baseRevision and payload. Create uses null base/live payload; update uses positive decimal base/live payload; delete uses decimal base/null payload. UUIDs are canonical lowercase hyphenated strings. Title has 1-200 Unicode scalars/non-whitespace content, notes 0-8,000, status open/in_progress/complete. NUL, unpaired surrogates, decoded duplicate keys and unknown fields fail validation. **[S1] [C1] [T1]**

Illustrative valid update; an existing live revision 4 is assumed

```
{
  "protocolVersion": 1,
  "mutationId": "20000000-0000-4000-8000-000000000001",
  "clientId": "10000000-0000-4000-8000-000000000001",
  "recordId": "30000000-0000-4000-8000-000000000001",
  "operation": "update",
  "baseRevision": "4",
  "payload": {
    "title": "Pump inspection",
    "status": "complete",
    "notes": "Valve checked"
  }
}
```

If it commits, the wrapper contains protocolVersion 1, that mutationId, replayed=false and committed outcome with committedRevision:"5" and the full record 5. A stale request instead records expectedRevision:"4" and remoteAtDecision at adjudication. These versions are STRINGS. Same-ID replay sets the response's replayed flag true while preserving the logical outcome; the stored response is not reconstructed from today's row. **[S2]**

Illustrative conforming terminal 409 response, not a captured request

```
{
  "protocolVersion": 1,
  "mutationId": "20000000-0000-4000-8000-000000000001",
  "replayed": false,
  "outcome": {
    "kind": "conflict",
    "reason": "stale_revision",
    "expectedRevision": "4",
    "remoteAtDecision": {
      "id": "30000000-0000-4000-8000-000000000001",
      "title": "Pump inspection",
      "status": "open",
      "notes": "Another inspector changed it",
      "revision": "5",
      "deleted": false
    }
  }
}
```

Other conflict reasons are record_missing (null remote), record_deleted and record_exists for a create colliding with live identity. Changed content under the same mutation ID returns mutation_identity_collision rather than a new adjudication. Admission errors include invalid_json, body_too_large, unsupported_media_type, invalid_envelope and origin_not_allowed. Internal errors or unreadable replies never authorize fabricated terminal conflicts. **[S1] [S3] [C3]**

Presentation is not another authority. Work phases are draft/queued/terminal_conflict; mutations additionally have outcome_unknown/acknowledged. Attempts/notBefore/errors can change while envelope/canonical/hash stay fixed. Frozen payload remains historical after active work is released or marked kept-server. **[C2] [C5]**

14 / Five complete code paths

Follow transactions, not only happy-path arrows

1 / Create, save, freeze, send, acknowledge

Typing changes React form state only. Save calls LocalStore.saveDraft with expected local version; parsing precedes its work/clean transaction. A new local-only draft retains null base/commonBase. Queue calls freeze: UUID/canonical/hash preparation happens outside the transaction, then current version/phase/slot is reread before mutation+work lock commit together. Failure leaves the prior saved state, not half a frozen request. **[C5] [C2] [C1]**

Send joins the running promise, checks durable integrity and commits markAttempt BEFORE deliver fetch. Raw media/UTF-8 admission and parseMutation precede SQL. processMutation claims identity, inserts 1, fills receipt and commits. validateReceipt checks ID/revision/deletion/data against the issued operation; applyReceipt commits acknowledgment/clean/work together. Only then can UI say acknowledged. Network ambiguity or local receipt abort retains original identity. **[C3] [S3] [S2] [C2]**

2 / Independent clients from the same base

A/B independently pull 1 and save different values. Neither save changes SQL. They freeze distinct IDs/base 1. F8 holds both workers before record locking; the chosen winner locks 1, conditionally commits 2 and receipt. The loser sees 2, commits a stale receipt without effect. Its base 1/frozen values/history remain; pull can refresh clean 2 without deleting work. Both chosen winner orders are tested. **[T12] [H2]**

3 / Commit, lost reply, process reopen, newer state

A route forwards to the actual backend, checks SQL receipt, then withholds/aborts delivery while local state is unknown. F9 closes the WHOLE Chromium process and reopens the SAME profile under a different PID, comparing exact ID/hash/envelope/lock. A later writer may advance state. Recovery sends the SAME ID before outcome inference, stores the original success and pulls latest. F1 proves receipt 2/clean 3; F9 proves receipt 6/clean 7. Neither snapshot equality nor a replacement request is recovery. **[E4] [E5] [C3]**

4 / Explicit keep or candidate, with a second conflict

Review latest precedes choice. keepServer confirms the matching active conflict/current observed revision inside a transaction, adds local decision metadata and removes unresolved work without POST. Or beginResolution validates selected values, captures live base/commonBase and writes editable work. Saving changes the candidate; cancel restores its predecessor. freeze associates a new ID/resolves and locks it. **[C6] [C2]**

A locally observed change before freeze forces review again, not silent rebase. A server-only change after review produces a normal second conflict at send. The latest active attempt becomes actionable while both earlier attempts remain historical. Another intentional candidate requires another ID/base; an unknown resolution never licenses a contradictory replacement. **[T11] [C2] [C5]**

5 / Deletion while another client is offline

queueDelete requires confirmation and matching observed live revision. Its envelope payload is null while commonBase/intent retain context. SQL locks/checks, marks deleted and increments without removing the row. An offline old-base update sees the retained tombstone on pull and gets record_deleted if submitted. Proposal survives, reapply/combine are unavailable and explicit keep deletion releases work. A stale delete against newer LIVE state may prepare another confirmed checked delete; against a tombstone no extra increment is created. **[C2] [S2] [C6] [T12]**

Source and evidence: **[T4] [T9] [T10] [T11] [T12] [E4] [E5]**

15 / Correctness and concurrency reasoning

What each invariant needs in order to mean anything

An invariant is a property the implementation must preserve, not a synonym for a passing test. Its assumptions matter. Serial duplicate requests cannot alone establish concurrent arbitration; a thrown exception cannot establish actual process exit; final equal caches cannot prove universal liveness. The repository gives each obligation a stable name so mechanism and evidence can be reviewed separately. **[D3] [D6]**

Invariant	Meaning / mechanism	Executed support and assumption
INV-ACK-01	Acknowledged server effect survives supported app restart; receipt/effect commit	F1/F6, T3; same retained functioning PostgreSQL
INV-ATOMIC-01	Record transition and terminal receipt share SQL transaction	T2/F2/F6; no out-of-band write path
INV-IDEMP-01	Same ID+bytes returns original outcome; mismatch collides	T1/T2/F1/F2/F9; lifetime receipt retention
INV-REV-01	Only matching live base advances, exactly once	F3/F7/F8; row lock+predicate, server revisions
INV-DRAFT-01	Committed intent survives supported restart/pull	T4/F5; same profile/origin, no eviction
INV-ACKCLIENT-01	Receipt/clean/work local transition is atomic	Native ack abort/F1/F9; functioning IDB
INV-UNKNOWN-01	Ambiguity retains original identity and remains unresolved	Pre-send marker/F1/F6/F9; future replay opportunity
INV-CONFLICT-01	Proposal/base/decision/current view remain explicit	F7/F8 and T11; user choices required
INV-TOMB-01	Retained deleted identity cannot silently reactivate	T2/F7/F10; no pruning or admin reset
INV-SNAP-01	Complete coherent read installs atomically/monotonically	T6/T10; valid full response and same DB incarnation
INV-ONE-01	One active issued operation per local record	Freeze/rechecks/F4/F9; one controller/profile
INV-RECON-01	Conditional progress to stable authoritative view	F4/F8/F10/demo; reachability, quiescence, user decisions

Same ID and different IDs are different races

For the SAME ID, unique receipt ownership prevents duplicate processing before record access. The waiting transaction may replay or take over after rollback. For DIFFERENT IDs/base-r, both are legitimate logical operations but only one can match the current row when it is adjudicated. The second gets a terminal conflict. Removing either receipt arbitration or revision gating leaves a different failure open. **[S2] [T2]**

Why check-then-write is not enough

Reading version r outside the transaction and later unconditionally writing is unsafe: another request can advance after the read. processMutation instead locks current row, checks base and retains the revision predicate in the UPDATE. A stale loser does not get to overwrite even if it was submitted earlier. This prevents that lost-update schedule; it does not promise multi-record serializability. **[S2] [T12]**

What READ COMMITTED does and does not do

A plain SELECT observes a statement snapshot. A later statement can see a committed uniqueness winner invisible to the earlier insert snapshot. Row-lock waits and rechecks are part of the concrete algorithm, not a claim that every statement in the transaction has one frozen worldview. Snapshot reads deliberately use one SELECT so their own complete array/count are coherent. **[S2] [S3]**

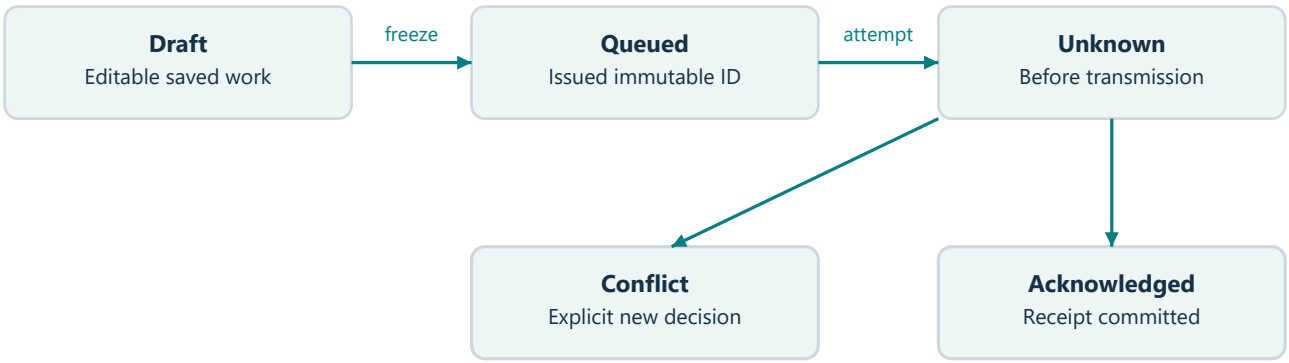
Why exactly-once delivery is the wrong claim

HTTP can duplicate or lose delivery; retries intentionally create more attempts. Under retained server receipts and unchanged canonical identity, the implementation records one terminal decision/effect for that logical operation. If receipts are pruned, the database is replaced or identity is lost, assumptions break. Say duplicate-safe replay under stated conditions, not exactly-once network delivery or universally lossless storage. **[D2] [D9]**

16 / Client transactions and crash recovery

Two stores, several crash windows, one retained identity

SQL and IndexedDB cannot share one distributed transaction in this application. Safety comes from committing enough local identity before transmission and making each local transition atomic. Preparation and storage have different lifetimes: generate/validate/hash before a write transaction, then reread/recheck the draft inside. Network waits cannot be held inside an IndexedDB transaction and still behave like a remote commit. **[C2] [C3]**



A deliberate resolution returns to a draft and later freezes a NEW mutation ID.

Crash or abort boundary	Committed local knowledge	Recovery rule
Before completed Save	Unsaved typing may exist only in UI	Do not promise recovery of uncommitted text
During freeze transaction	Earlier draft OR full frozen request+lock	Abort leaves no partial queue/lock
After freeze, before attempt	Issued request queued; no attempt marker	Send exact issued bytes when requested
After unknown marker, before fetch	Possibly transmitted identity retained	Replay even if it might never have reached SQL
After SQL commit, before local receipt	Server receipt may exist; client unknown	Same-ID replay, never snapshot-based guessing
During acknowledgment transaction	Previous unknown OR complete terminal transition	Abort retains original eligible identity
After terminal conflict	Decision final, proposal still unresolved	Confirm keep or create a deliberate new attempt

The pre-send native-abort browser case verifies zero transport calls when committing the unknown marker fails. A different native receipt-put abort occurs after a real SQL commit and must leave unknown eligible. These are separate failure locations with different server knowledge. Both preserve identity; neither proves behavior under broken storage hardware. **[T12] [T9]**

A stored fingerprint mismatch blocks transport; malformed durable content can also surface a foreground error. The implementation does not repair corrupted identity by editing or re-IDing it. Recovery assumptions include retained functioning local storage. Profile deletion loses the information the protocol needs, and current server content cannot reconstruct an exact forgotten operation. **[C3] [T7] [D9]**

17 / Reconciliation and conflict semantics

Fresh observations do not rewrite old intentions

Conceptual sequence implemented by PushController.synchronize

```
recover unknown identities
pull one complete authoritative snapshot
apply snapshot atomically without touching pending intent
send genuinely queued immutable operations
pull and apply another complete snapshot
report completed observation, with remaining work visible
```

Recovery attempts happen first. If they exhaust the budget, the engine may still refresh separate clean rows while KEEPING those operations unknown; it never classifies their outcomes from that snapshot. A terminal conflict on the selected record and an unknown operation on another record are different work. The shared foreground guard serializes calls, but supported usage still has one controller per profile rather than competing tabs. **[C3] [C2]**

Snapshot completeness means every authoritative row, including tombstones, from one statement. It is not a global synchronization barrier. Local installation validates everything before writes and checks all known authoritative IDs inside the transaction. Missing known rows signal incompatibility/reset, not permission to erase them. A local-only create absent remotely is not yet an authoritative known row, so it remains separate pending work. **[C4] [C2] [S3]**

A draft keeps the original base even after clean state advances. Replacing it would silently authorize overwriting changes the user had not reviewed when authoring. Resolution is the explicit moment to choose reviewed current values/base for a NEW immutable request. A pull cannot make an old terminal receipt current, and a historical success cannot roll a newer known clean row back. **[C2] [C6] [E4] [E5]**

Conditional convergence means that, after finite writes stop, functioning retained clients with repeated reachable foreground opportunities and resolved conflicts can install the stable server view. Endless writers, permanent disconnection, blocked corruption or undecided proposals can prevent completion. The demo provides a bounded successful instance under these conditions, not a universal liveness proof. **[D3] [T13]**

Full snapshots intentionally avoid a durable change log/cursor/retention protocol. A future incremental path would need ordering, checkpoints, safe tombstone/history retention, cursor persistence and a full-reset procedure that preserves pending work. Adding a timestamp filter alone would not solve those obligations. No expired-cursor behavior is currently implemented or tested. **[D2] [D5]**

18 / Failure-injection atlas

Initial condition, exact fault, recovery and evidence limits

The table condenses existing F1-F10 contracts and inspected Phase 7 artifacts. PASS means the bounded named tests ran previously and their retained observations were inspected here; it does NOT mean they were rerun for this PDF. Exact test titles are stored in release-check.json under each scenario. The source path is shown with each row and the reference index makes the current artifacts locatable. **[D3] [E1]**

ID	Initial state / injection	Expected and observed / recovery	Invariant / executed path / limit
F1	clean 1, immutable update/base 1, no receipt. Fault: commit 2 held before browser delivery; reload destroys old controller.	Client: unknown/locked original bytes; replay before pull gives historical 2 and clean 3. SQL: one original receipt, final3. Recovery: same-ID replay then full pull.	INV-ACK-01, INV-ATOMIC-01, INV-IDEMP-01, INV-UNKNOWN-01, INV-ACKCLIENT-01. tests/browser/audit.spec.ts. PASS (prior execution inspected). Interception/reload; local ack abort additionally covered in resolution/browser regressions.
F2	live1 or live2, no target receipt. Fault: hold owner claim, observe independent waiter PostgreSQL lock; duplicate and collision HTTP.	Client: HTTP client sees original decision and replay; collision explicit. SQL: one target receipt; final2, conflict adds no effect. Recovery: identical sequential/concurrent replay; owner-rollback variant in backend regression.	INV-IDEMP-01, INV-ATOMIC-01, INV-REV-01. tests/audit.test.ts. PASS (prior execution inspected). HTTP clients plus existing browser collision regression; no exactly-once delivery claim.
F3	two isolated profiles at 1 with distinct update IDs/base 1. Fault: hold first-sent HTTP before admission; deliver other first, both orderings.	Client: one acknowledged, one preserved terminal conflict/base 1/local values. SQL: one effect to 2, success and conflict receipts. Recovery: explicit keep-server and both final reconcile.	INV-REV-01, INV-CONFLICT-01, INV-IDEMP-01. tests/browser/audit.spec.ts. PASS (prior execution inspected). Controlled finite orderings; no general network ordering guarantee.
F4	three independent queued creates, no target receipts. Fault: first acknowledged; second commit reply held until reload; third unsent.	Client: acknowledged/unknown/queued; final all acknowledged with same IDs. SQL: two receipts before recovery, three after; each record 1. Recovery: unknown first, full pull, send queued third, final pull.	INV-ATOMIC-01, INV-UNKNOWN-01, INV-ACKCLIENT-01, INV-ONE-01, INV-SNAP-01, INV-RECON-01. tests/browser/audit.spec.ts. PASS (prior execution inspected). No multi-record atomicity; snapshot abort/truncation covered by existing snapshot suite.
F5	persistent profile with saved draft and separate offline queued mutation. Fault: graceful whole Chromium close/relaunch; later unknown variant and second acknowledged reopen.	Client: exact durable identity/work/ID/hash/envelope/lock across launches; fresh profile empty. SQL: no effect offline; one receipt per sent operation. Recovery: same retained profile, exact retry, full reconciliation.	INV-DRAFT-01, INV-ONE-01, INV-UNKNOWN-01, INV-ACKCLIENT-01. tests/browser/audit.spec.ts. PASS (prior execution inspected). Graceful only, reachable frontend shell, no eviction/forced browser kill/power loss.
F6	local frozen create marked unknown; no server record/receipt. Fault: actual dedicated Node SIGKILL after provisional write or after commit, before reply.	Client: transport lost, unknown original lock retained; local acknowledgment after retry. SQL: before kill: 0/0 or record 1/receipt 1; after retry always1/1. Recovery: new Node PID same database/schema; original ID executes or replays.	INV-ACK-01, INV-ATOMIC-01, INV-IDEMP-01, INV-UNKNOWN-01. tests/process.test.ts. PASS (prior execution inspected). Actual Node termination; local store fake-indexeddb, separate actual-browser coverage; PostgreSQL remains running.

ID	Initial state / injection	Expected and observed / recovery	Invariant / executed path / limit
F7	browser original base 1, server 2, no stale-request receipt. Fault: stale update/delete; durable conflict2 reply lost; server later3 then tombstone 4.	Client: original intent/base and historical conflict2 retained; current clean 3/4 separate. SQL: stale requests never change server; create/update/delete cannot revive tombstone 4. Recovery: same-ID conflict replay, pull, explicit keep deletion.	INV-REV-01, INV-CONFLICT-01, INV-IDEMP-01, INV-UNKNOWN-01, INV-TOMB-01. tests/browser/audit.spec.ts. PASS (prior execution inspected). Receipt/tombstone retention within unpruned test DB, no restore coverage.
F8	independent profiles at 1, different IDs/client IDs. Fault: both HTTP transactions held before_record_lock; controlled A-first and B-first releases.	Client: one acknowledgment and visible preserved conflict until deliberate new resolution. SQL: one transition to 2; two original terminal receipts; resolution to 3. Recovery: new ID/base 2 then both clean caches3 under quiescence.	INV-REV-01, INV-CONFLICT-01, INV-RECON-01. tests/browser/audit.spec.ts. PASS (prior execution inspected). One controller per independent store; no shared-profile tab arbitration.
F9	genuine conflict/base 4 at server 5, explicit new resolution/base 5. Fault: real commit 6 response dropped; graceful whole Chromium restart; writer 7 before replay.	Client: same unknown ID/hash/envelope/lock/history; original resolution receipt 6 then clean 7. SQL: one resolution receipt/effect; final7. Recovery: same-ID replay before pull; local ack abort separately rerun in same spec.	INV-IDEMP-01, INV-UNKNOWN-01, INV-ACKCLIENT-01, INV-CONFLICT-01, INV-ONE-01. tests/browser/resolution.spec.ts. PASS (prior execution inspected). Graceful browser restart, no forced browser kill or power loss.
F10	old-base queued record and separate previously committed unknown record. Fault: API disconnected; simulated wall clock +21 days; remote update/delete.	Client: unknown replay first, tombstones observed, stale proposal retained until keep-server. SQL: tombstones3 and 2, no resurrection or unintended increment. Recovery: recover unknown, pull, stale request conflicts, explicit keep, both caches equal.	INV-TOMB-01, INV-DRAFT-01, INV-UNKNOWN-01, INV-CONFLICT-01, INV-SNAP-01, INV-RECON-01. tests/browser/audit.spec.ts. PASS (prior execution inspected). Simulated duration only; independent not-before fixture schedules queued work, no cursor/aging claim.

For process evidence, inspect actual identifiers rather than only a test name. Current F5 records four graceful Chromium PIDs 10440,13972,38256,32400. F9 records 44048 to 42476. Actual forced Node precommit 25132 to 44488 and postcommit 4792 to 28036 are separate runs. These are inspected artifact observations, not newly launched processes in this documentation task. **[E6] [E5] [E7] [E8]**

The atlas's supplemental variants remain important: malformed/truncated snapshots, missing known tombstones, delayed lower snapshots, native freeze/ack/marker abort, canceled choices and second conflicts. A browser exception is not real storage exhaustion; a virtual 21-day clock is not physical retention; Node restart is not PostgreSQL restart. Eviction, forced browser kill and OS/power loss remain unverified. **[T6] [T10] [T11] [D9]**

19 / Performance and complexity

Use numbers only inside their measurement contract

Phase 7's raw file contains 4,010 timed observations from two repetitions, not 4,010 new correctness cases. Seed 20261105 fixes synthetic values/record IDs; browser freeze still uses real random UUIDs. Fixtures are isolated generated schemas with N=100/500/1000 and 10% tombstones. HTTP groups use five warmups and 100 measured samples per repetition; browser installation uses three and 50. Full foreground cycles use one warmup and five each. **[E2] [U2]**

Measured boundary	Pooled n	Median ms	Empirical p95 ms
Accepted HTTP	200	4.995	7.422
Receipt replay	200	3.401	4.617
Stale conflict	200	4.156	6.360
Browser applySnapshot N=100	100	28.550	44.800
Browser applySnapshot N=500	100	126.050	177.700
Browser applySnapshot N=1000	100	286.350	478.600
Foreground N=100, Q=10	10	280.200	Not reported
Foreground N=500, Q=50	10	1250.750	Not reported
Foreground N=1000, Q=100	10	2193.800	Not reported

Direct HTTP uses Node performance.now from fetch start through complete response body; request preparation and client JSON parsing are excluded. It includes server admission/hash/SQL commit/serialization. Browser applySnapshot uses browser performance.now around validation, known-row checks and atomic IndexedDB work, excluding network/DOM. Full synchronize includes Vite proxy, one original-ID recovery, queued writes and two complete installs; initial save/freeze and first unknown commit preparation are excluded. These are not interchangeable latency measurements. **[E2] [D8]**

Snapshots 100/500/1000 measured 45,306/227,360/455,017 uncompressed body bytes; direct GET medians 2.834/8.581/14.038 ms with n=200 each. At 1000, validate-only median 20.750 ms is distinct from combined install 286.350 ms. Existing rows are already known after warmup; this is not empty-store or cold-start timing. **[E2]**

Distinct-record writer widths 1/2/10/20 measured 135.7/208.9/372.8/509.5 requests/s in repetition 1 and 235.3/359.7/535.0/446.9 in repetition 2,100 accepted requests each. Pool max 12 means width 20 can queue. Same-record races separately have one winner/round and expected 0/50/90/95% conflicts. Repetition variation and different race clock boundaries prevent a universal scaling or causal ranking. **[E2] [D8]**

The benchmark host was Windows build 10.0.26200 x64, Node 24.15.0/npm 11.12.1, PostgreSQL 18.6 UTF-8/READ COMMITTED with fsync/synchronous_commit/full_page_writes on, Chromium 156.0.8078.4; Ryzen AI 9 HX 370, 12 cores/24 logical,24,823,836,672 RAM bytes and NVMe SSD. Caches were warm, processes co-located, background load not controlled. Median is the middle; p95 nearest rank ceil(.95*n), omitted below 100. Foreground intermediate retry counts were not instrumented. **[D8] [E2]**

Cost / retained quantity	Reasoned model	What wall time also depends on
Canonical/hash	O(B) valid bytes	Encoding/copy, runtime, payload distribution
Complete snapshot	O(N) records, O(NB) bytes/materialization	SQL sort/encoding, two pulls, IDB puts
Pending retrieval	About O(Q) returned data; generic sort O(Q log Q)	Phase index, materialization, retained sizes
Record/receipt lookup	B-tree size/cache/plan-dependent	Row/uniqueness waits and durable commit
Lifetime storage	Unbounded retained N/M/H	Receipts duplicate request/response; dead tuples/pages

Actual SQL growth with 100 seeded records reached 2,834,432 allocated receipt-relation bytes at 1000 receipts in both repetitions; ten more deletes increased logical receipts to 1010 and tombstones to 20 without another allocation step. That does not mean zero cost: existing page capacity was reused. pg_total_relation_size includes indexes/TOAST/allocation/dead versions, not a linear forecast. Snapshot/receipt EXPLAIN results are retained, not replaced by unconditional O(1) claims. **[E2]**

Potential bottlenecks include repeated full transfer and per-row IDB writes, but N/Q co-vary in foreground tests and replay uses a different fixed smaller request from accepted/conflict payload cycles. Observed differences cannot isolate one cause. No performance defect justified production optimization. WAN/mobile/cold-start, sustained load, arbitrary scale and third-party benchmarks were not measured. **[D8]**

20 / Security, operation and future alternatives

Technical review readiness is not permission to publish

The application is unauthenticated and loopback-only. Origin checks constrain browser requests, not local-process authority or tenants. The helper database uses local trust; only synthetic data belongs there. Client IDs and hashes are tracing/integrity inputs, not credentials. Exposing a public port/proxy would require a separate security design, not a one-line deployment tweak. **[S5] [U1] [D9]**

Existing process hooks require explicit SYNCPROOF_TEST_PROCESS=1, NODE_ENV=test and parent IPC. Negative startup cases and HTTP crash-route/header probes passed in prior release checks. Phase 7 output routing only changes test destinations; it is not a runtime fault switch. A heuristic candidate scan found no token/key/password-shaped finding, but that is not a comprehensive secret or application security audit. **[S5] [T3] [E12]**

At the recorded review, npm audit reported zero known advisories;119 installed manifests had known license metadata. Three lacked detected top-level notice filenames and platform-optional uninstalled entries were metadata-only. No project license was chosen. Historical absolute private paths still require deliberate review/redaction before public sharing. No commit, push, deployment, license decision or external peer review is established. **[E11] [E12] [E13] [D9]**

- Before public sharing: choose license/rights/notice handling, review staged files and then-current history, review private paths, recheck advisories and obtain separate approval.

- Before changing retention: design how old clients recover without forgotten receipts/tombstones; do not prune by a convenient age alone.

- Before adding incremental sync: define ordered changes, checkpoints, retention, cursor persistence and safe full-reset with pending work.

- Before supporting multiple controllers: add a real shared-store coordination protocol and targeted races; do not assume disabled buttons arbitrate tabs.

Storage eviction, profile deletion, private-profile lifetime, forced browser kill, OS/power/hardware/database-process failure, replaced/restored database incarnation, public multi-tenant use and multi-region are not supported guarantees. Permanent partitions or endless writers can prevent progress. These are explicit MVP limits, not flaws to hide with the word offline-first. **[D2] [D9]**

IV / Defend and extend it independently

Use reproducible observations, accurate attribution and self-authored exercises. A supplied explanation is a starting point, not evidence of mastery.

21 / Run and reproduce safely

A verified runbook, not commands executed by this guide

The following commands come from the current README/scripts and were supported by prior Windows validation. They are instructions for a later owner run, not actions performed to create this PDF. Use the repo root, Node 24 and a dedicated UTF-8 test database. The helper assumes installed PostgreSQL 18 binaries unless PostgresBin is supplied. Other operating systems were not certified. **[D10] [U1] [E14]**

PowerShell: dependencies and browser setup, with actual script names

```
npm ci --ignore-scripts
npm run build
$env:PLAYWRIGHT_BROWSERS_PATH=(Join-Path (Get-Location) '.local/browsers')
npx playwright install chromium
```

PowerShell: local manual demonstration; synthetic data only

```
# Terminal 1: project-owned database and normal backend
.\scripts\local-postgres.ps1 -Action Start
$env:DATABASE_URL='postgresql://syncproof@127.0.0.1:55432/syncproof_dev'
$env:ALLOWED_ORIGIN='http://127.0.0.1:5174'
$env:PORT='3000'
npm run migrate
npm start

# Terminal 2: frontend
npm run dev:client

# Terminal 3: two separate retained owner profiles
node scripts/demo-clients.mjs
```

Open 127.0.0.1:5174, Save draft, Queue for future sync, Send pending / Retry and Reconcile now. Default owner profiles persist after close; two tabs are not independent. Keep the same database incarnation when reusing them. Do not clear profiles or developer tables merely to hide missing-known-ID errors. Stop normal servers before test/benchmark commands own 3000/5174. **[D10] [U4]**

PowerShell: supported checks; do not add overlapping runs as distinct cases

```
$env:TEST_DATABASE_URL='postgresql://syncproof@127.0.0.1:55432/syncproof_test'
$env:SYNCPROOF_EVIDENCE_PHASE='phase-7'
$env:SYNCPROOF_AUDIT_RUN_ID=[guid]::NewGuid().ToString()
npm test
npm run test:local
npm run test:push
npm run test:snapshot
npm run test:resolution
npm run test:audit:db
npm run test:browser
npm run typecheck
npm run build:client

# Focused audit / ordered demo (some cases overlap the full run)
npm run test:audit
npm run test:demo
```

PowerShell: measurements/review; read boundaries and preserve prior outputs

```
node scripts/bench.mjs --smoke --seed=20261105
node scripts/bench.mjs --seed=20261105
node scripts/reproduce.mjs --online
node scripts/demo-clients.mjs --check
npm audit --json
node scripts/release-audit.mjs

# When finished, stop ONLY the owned cluster
.\scripts\local-postgres.ps1 -Action Stop
```

For a fully recaptured evidence audit, retain the exact log filenames and matching run-context required by verify-release.mjs; ordinary test execution does not automatically create every transcript. Existing JSON/output names are latest-run artifacts, so archive old evidence separately before another audit. The current verification command needs PostgreSQL available for cleanup/settings checks. Reading this guide requires none of those services running. **[U6] [P7]**

First 30 minutes: suggested study route

Minutes	Action	Check yourself
0-5	Read ownership/glossary and App actions	Separate server truth, draft and historical receipt
5-10	Trace parseMutation and processMutation	Locate identity claim and COMMIT
10-15	Trace saveDraft/freeze/markAttempt	Name local atomic boundaries
15-20	Read current F1/F9 artifacts	Explain old receipt versus current revision
20-25	Follow one ordered demo trace	Find all eight steps, not isolated substitutes
25-30	Attempt one question/exercise unaided	State one assumption and an untested modality

Troubleshooting decision tree

- Connection refused to 55432? Check whether the owned helper is running and the intended URL is set. A stopped database is setup failure, not a stale conflict. Never redirect tests at user data to get green output.

- Port 3000/5174 occupied? Stop only your normal demo/test processes or choose the documented workflow; do not kill arbitrary system processes.

- Chromium executable missing? Set the project browser path before import/install and use the pinned Playwright toolchain. Phase 7 smoke documented this discovery-order issue.

- Offline npm cache miss? An offline install cannot download a missing tarball. The recorded explicit network-enabled lockfile retry passed; do not change dependency versions just to suppress the error.

- Snapshot missing known ID / lower revision? Verify database incarnation and payload completeness. Fail closed; absence is not permission to delete local knowledge.

- Unknown remains after retries? Inspect original ID/hash/notBefore/block diagnostics and connectivity. Do not replace identity or infer outcome from current content.

- Conflict reappears after resolution? Inspect the NEW attempt/base and the later writer. This can be correct optimistic concurrency, not necessarily a bug.

22 / Interview questions and adversarial follow-ups

Answer from code and evidence, not slogans

Each outline is a starting point. A strong answer names a mechanism, a concrete schedule and a limitation. Try answering first, then locate its source. None of these model answers certifies the owner's independent understanding.

Foundations

1. What problem does SyncProof actually solve? A bounded local demonstration of durable intent, checked writes and ambiguous acknowledgment recovery; not a production replication platform. Distinguish the educational workflow from its unsupported operating models. **[D1] [D9]**

2. Why are record ID, mutation ID and revision separate? Record identity spans many operations; one mutation ID fixes one immutable attempt; revision versions one server row. Deduplicating by record ID would suppress legitimate later edits. **[S1] [S2]**

3. What does atomicity mean on each side? SQL couples effect/receipt; IndexedDB couples local state transitions. There is no shared cross-network transaction, which is precisely why unknown and replay exist. **[S2] [C2]**

4. When can clients converge? Under finite/quiescent writes, retained functioning stores, repeated reachable opportunities and human conflict decisions. A passing bounded demo is evidence of that instance, not unconditional liveness. **[D3] [T13]**

Implementation

5. Why canonical bytes and hash? Defined encoding removes irrelevant JSON ordering/whitespace while preserving business differences; server compares bytes and fingerprint rather than assuming hash alone establishes authorization or perfect identity. **[S1] [S2]**

6. Why detect duplicate decoded JSON keys? Ordinary object parsing can discard a duplicate or escaped alias. The custom parser rejects ambiguity before that information is lost. Name an actual contract case. **[S1] [T1]**

7. How are Unicode and large revisions handled? Validated Unicode scalar rules reject NUL/unpaired surrogates without normalization. Decimal BIGINT strings avoid JS rounding; BigInt comparisons avoid lexical ordering errors. **[S1] [C1]**

8. What happens on identity collision? Changed immutable content under an existing ID returns explicit collision without altering the original decision. Client blocks with retained intent rather than auto-repairing by a new ID. **[S2] [C3]**

Concurrency

9. How do same-ID contenders arbitrate? Receipt uniqueness is claimed before record work. Waiter observes owner commit and replays, or may own processing after rollback; actual backend blocking is tested. **[T2] [S2]**

10. Why row lock plus a predicate? Lock/read/adjudicate occur within the write transaction, and UPDATE still checks expected revision. A stale external check followed by unconditional write would leave a race. **[S2]**

11. Why test two delivery orders and both winners? Safety must not depend on transport order or which legitimate request reaches row adjudication first. F3/F8 preserve loser evidence in each controlled ordering. **[T12]**

12. Can disjoint field edits conflict? Yes. Whole-record expected revision rejects stale replacement even when humans see disjoint changes. The explicit manual candidate chooses combined values; no automatic merge is implemented. **[C6]** **[T11]**

Failure recovery

13. Why persist unknown before fetch? If transmission precedes durable identity, a crash could forget a committed operation. A failed marker must prevent transport, which the native-abort case asserts. **[C3]** **[T12]**

14. Why is a timeout not rejection? The request may commit while its response is lost. Only a valid original receipt settles that operation; another retry uses the exact same ID and bytes. **[E4]** **[C3]**

15. Why recover before interpreting a newer snapshot? A later write can follow the client's successful unacknowledged write. Receipt 2 and current 3 can both be correct; current equality/difference cannot identify history. **[E4]**

16. What if acknowledgment storage fails? The local transaction rolls back receipt/clean/work together, leaving original unknown recoverable. SQL may already have committed. Retry and atomic install, not a replacement mutation. **[T9]** **[C2]**

Database

17. Why READ COMMITTED matters here? Statements can see different committed snapshots. A fresh read after uniqueness arbitration can observe a winner absent from an earlier insert snapshot; row-lock/revision checks still protect writes. **[S2]**

18. Why receipt a terminal conflict? The original rejected operation needs stable historical replay even after later remote writes. Re-adjudication would make the same ID mean different decisions. **[S2]** **[E1]**

19. Why no receipt-to-record foreign key? A valid request to a missing record must still persist a record_missing conflict receipt. Requiring an existing inspection would prevent that terminal evidence. **[S6]**

20. What does the real Node kill establish? At controlled pre/postcommit boundaries the actual child exits and a new PID retries against retained PostgreSQL. It is not a PostgreSQL, OS or power-failure experiment. **[E7]** **[E8]**

Client and browser

21. Why crypto outside the Dexie transaction? Unrelated awaits can let IndexedDB become inactive. Prepare first, then re-read version/slot inside the atomic write to reject a race during preparation. **[C2]**

22. Why separate cleanRecords and work? Pull may advance authoritative observation without changing the user's original base/proposal or issued content. They represent different facts, not two copies to overwrite together. **[C2]** **[C4]**

23. Does a disabled button enforce one outstanding request? No. Transaction checks and a shared controller guard provide actual safety for supported usage. Competing tabs remain outside the promised model. **[C2]** **[C3]**

24. What does graceful Chromium reopen prove? Exact committed data and locks survived that same-profile/origin lifecycle under tested settings. Eviction, forced kill, physical weeks and power failure remain distinct untested models. **[E6]**

Design and tradeoffs

25. Why retain tombstones? Old clients need an explicit authoritative deletion signal and reserved identity. Missing known IDs are incompatible completeness, not an implicit delete or permission to restore. **[C4]** **[S2]**

26. Why can resolution itself conflict? Review is not a lock. Another write may advance before the new checked mutation; preserve the new rejection and original provenance, then choose another deliberate attempt. **[T11]**

27. Why full snapshots rather than cursors? They reduce protocol/retention complexity at O(NB) repeated transfer and installation cost. Incremental sync needs logs, ordering, cursor retention and safe reset semantics. **[D5]**

28. Is Origin a security identity? No. It constrains browser-origin access, not local processes or tenant authorization. The demo is unauthenticated loopback with synthetic data, not a hardened public service. **[S5]** **[D9]**

Evaluation and attribution

29. Why are 123 cases and 126 executions both correct? Three process cases run in overlapping commands. Named cases and repeated executions differ; fresh-copy checks and timing samples are not added as new correctness cases. **[E1]**

30. Why no foreground p95? Only 10 full-cycle samples were retained. The methodology omits p95 below 100 and preserves medians/raw values rather than presenting an unstable tail as an SLA. **[E2]**

31. Which benchmark comparison is confounded? N and Q increase together in full cycles; replay uses a different fixed smaller body; throughput varies across reps. Separate clocks and uncontrolled load prevent causal/general claims. **[D8]**

32. What can you honestly claim about your role? The sessions establish owner-directed AI-assisted development and retained evidence, not independent manual implementation or mastery. Describe your actual review/reproduction work only once you have done it. **[P7]**

Ten adversarial follow-ups

What if a receipt was pruned while the original unknown client was offline? Identify the assumption that failed before proposing a TTL.

Can you show that the dropped response followed a real COMMIT, rather than merely an aborted request? Point to SQL evidence.

If current state equals your desired payload, why is that not proof your operation committed? Give two possible histories.

What if a browser loses all IndexedDB but PostgreSQL still has receipts? Explain why the protocol cannot reconstruct a forgotten exact identity.

If both requests use the same record ID but different mutation IDs, which lock solves the race and which uniqueness rule does not?

If the claim owner rolls back, why is waiting forever or replaying an incomplete row wrong? Locate the takeover test.

Why does a retained tombstone conflict with a new delete ID even when its base matches? Distinguish replay from another operation.

What if the response contains an older success after you already know a newer clean row? Show the actual monotone application rule.

How would you justify a queue-speed optimization without mixing different N values or changing the failure contract?

Which observations can you reproduce unaided today, and which claims still come only from inspected Codex artifacts?

23 / A ten-session independent learning plan

Small changes, precise self-checks

Each session has one concept, one bounded exercise and a falsifiable self-check. Read the listed one-to-three entry files first. Do not silently execute these as part of this guide task; they belong to later owner work.

1 / Ownership

Read: README.md; client/main.tsx. Draw server/current/draft/receipt lanes for an unfamiliar mutation selected from an artifact. Self-check: distinguish each identity and classify every visible label without assuming server acceptance.

2 / Contracts

Read: src/contracts.ts; tests/contracts.test.ts. Write a decoded-duplicate or Unicode case without weakening validation. Self-check: predict admission status and explain whether canonical content changed.

3 / SQL transactions

Read: src/mutations.ts; migrations/001_mutation_server.sql; tests/integration.test.ts. Annotate claim/lock/receipt/COMMIT, then vary owner rollback in a copied test. Self-check: one effect/decision or takeover, with actual blocker evidence.

4 / Local atomicity

Read: client/lib/storage.ts; tests/local.test.ts. Inject freeze failure after one staged write in a self-authored case. Self-check: exact prior draft, no partial queue/lock; explain crypto preparation.

5 / Uncertainty

Read: client/lib/push.ts; tests/push.test.ts; tests/browser/push.spec.ts. Trace a new lost-reply variant using the same issued ID. Self-check: marker precedes network and only original receipt settles the outcome.

6 / Safe reads

Read: client/lib/pull.ts; client/lib/storage.ts; tests/browser/snapshot.spec.ts. Omit a known tombstone from an otherwise complete response. Self-check: all clean/work/history/metadata unchanged and later valid response succeeds.

7 / Repeated conflict

Read: client/conflict.tsx; client/lib/storage.ts; tests/browser/resolution.spec.ts. Race a reviewed base 5 candidate with writer 6, then choose a new base 6 candidate. Self-check: two rejections preserved, newest actionable, no old ID modified.

8 / Fault control

Read: tests/browser/audit-harness.ts; tests/browser/audit.spec.ts. Change one gate placement and explain the newly tested boundary. Self-check: no sleeps select the winner; SQL and durable client assertions prove the fault.

9 / Measurements

Read: scripts/bench.mjs; docs/PERFORMANCE_REPORT.md. Recompute a raw group and design a controlled one-dimension comparison. Self-check: correct n/median/nearest-rank p95 or omission, precise included/excluded work.

10 / Defend unaided

Read: tests/browser/demo.spec.ts; docs/PROTOCOL_SPEC.md. Write your own regression, run a full story, then narrate without looking. Self-check: retained real output, invariant mapping, one honest limit and accurate authorship attribution.

Completion is not hours spent or files skimmed. It is being able to predict a new schedule, write a test that would fail for the wrong implementation and explain what its passing result still cannot prove. **[D3]**

24 / Resume and demonstration guidance

Make the claim no larger than your evidence or role

The visible work establishes owner-directed, AI-assisted implementation and review artifacts. It does not establish that the owner independently wrote every line, ran every test or can yet defend all mechanisms. Do not remove that distinction to make a resume sound stronger. Adapt the following templates only to work you can personally explain and substantiate. **[P7]**

- Cautious current template: Coordinated an AI-assisted eight-phase local synchronization project using React/Dexie, Node and PostgreSQL, with immutable request receipts and explicit conflict decisions.

- After independent reproduction: Reproduced bounded duplicate/reorder/restart scenarios for a local sync demo and explained original-ID recovery, transaction boundaries and retained tombstones. Use only after doing that work.

- After independent analysis: Analyzed seeded localhost HTTP/IndexedDB/reconciliation measurements with raw sample sizes and stated limits. Do not imply WAN capacity, universal reliability or independent unassisted implementation.

A 60-second explanation

SyncProof studies a request that may commit even when its caller sees failure. The browser durably saves and freezes an immutable operation, marks it unknown before transmission and retries its original identity. PostgreSQL commits the checked record change and terminal receipt together. Same-ID replay reveals history, while a complete snapshot supplies current state. Concurrent stale work is preserved for explicit new-ID resolution, and deletions retain tombstones. The evidence covers controlled SQL/browser/Node schedules, not eviction, power failure or a public production service.

A 5-minute technical walkthrough

- Minute 1: show ownership and envelope; distinguish record ID, mutation ID and revision.

- Minute 2: open processMutation at unique claim, row check and shared effect/receipt COMMIT.

- Minute 3: show freeze/markAttempt/applyReceipt and the two separate transaction systems.

- Minute 4: show historical receipt versus latest clean row and a new resolution identity.

- Minute 5: inspect one actual F artifact/ordered trace, then state an unsupported failure and a benchmark confounder.

A 10-minute adversarial demo outline

Prepare the supported local environment before the interview. Use two truly independent profiles or the existing ordered test, then narrate common 1, winner 2, preserved stale conflict, explicit resolution 3, separate committed/lost create, exact replay, distinct offline queue surviving actual graceful browser restart and final equality with a tombstone. Show SQL receipt counts and durable trace state rather than only UI color. End by distinguishing forced Node exit, graceful Chromium restart, simulated 21 days and physical failure. These minute labels are presentation planning, not a newly measured demo duration. **[T13] [E9] [E10]**

Claims not to make

Do not say zero data loss, exactly-once network delivery, formally verified, always consistent, production-ready, safe public exposure, robust under storage eviction/power loss, or a replacement for production replication tools. Do not sum repeated suites or timing observations as extra correctness tests. Do not claim independent implementation, user mastery, external review, screenshots or deployment that did not happen. **[D9] [P7]**

25 / Final audit and twelve things to remember

What was inspected, what was reproduced and what stays open

Implemented: the bounded Phase 0-5 workflow. Reported and artifact-supported: the later fault audit, ordered demos, release checks and warm measurements. Independently checked in THIS documentation session: current source hashes against the 45-file Phase 7 record, phase gate/test-count provenance, retained artifact structure, benchmark arithmetic, source/path references and the PDF's content/layout. No database, browser application test suite or benchmark was rerun here. Reading a PASS artifact is not re-executing its test. **[E1] [E2] [E15]**

No material source drift was found against the recorded hashes. Historical report statements such as GET being deferred in Phase 1 or the full demo unrun in Phase 5 are chronological milestones, not claims about the final product. Phase 7's PASS means local deliverables ready for owner review; it coexists with a no-go for immediate public sharing pending license/private-path decisions. That distinction is preserved here rather than choosing the more flattering label. **[P1] [P5] [P7] [D9]**

Phase	Primary report	Key retained evidence
0	**[P0]**	Specification-only 14 gates; no runtime execution
1	**[P1]**	**[E17]**:18+27; **[T3]** actual Node boundaries
2	**[P2]**	**[E18] [E19]**:13 local,4 actual browser
3	**[P3]**	**[E20]**:15 browser plus 9 push
4	**[P4]**	**[E21]**:7 snapshot browser,4 local snapshot
5	**[P5]**	**[E22]**:31 browser total,5 resolution
6	**[P6]**	**[E16]** and ordered full demo **[T13]**
7	**[P7]**	**[E1] [E2] [E9] [E10] [E11] [E14]**

Document quality-control checklist

QC	Result / criterion
1	PASS - All eight phases have dedicated substantive chapters and the required phase categories.
2	PASS - Final architecture, critical files and actual contracts are mapped.
3	PASS - Five code paths trace storage/SQL/request boundaries and recovery.
4	PASS - Every core invariant and F scenario has source/test/evidence or explicit limits.
5	PASS - Prior execution is distinguished from newly performed documentation validation.
6	PASS - Benchmark values, n, clocks, warmup and limitations match inspected raw artifacts.
7	PASS - Graceful/forced/simulated modalities and unsupported eviction/OS failure remain separate.
8	PASS - Cross-phase continuity adds synthesis rather than merely copying the existing handbook.
9	PASS - Internal PDF destinations and referenced repository paths are checked.
10	PASS - Runbook,32 model answers,10 follow-ups, exercises and honest resume guidance are included.
11	PASS - No production source, dependencies, protocol, migrations, tests or old evidence were modified.
12	PASS - No Git operations, deployment, public sharing or new implementation phase occurred.

If you remember only twelve things

1. A record, an operation and a revision have different identities.
2. A saved local draft is not a server acknowledgment.
3. Freeze atomically commits issued content and its lock.
4. Unknown must be durable before transmission.
5. Silence cannot tell whether a server committed.
6. The record effect and original terminal receipt share one SQL commit.
7. Same-ID replay is history; a snapshot is current observation.
8. Different same-base IDs race through the revision gate.
9. A resolution needs a new ID and can conflict again.
10. Tombstones preserve deletion; missing known IDs are not deletes.
11. Real process restart, simulated failure and physical loss are different claims.
12. Convergence, performance and authorship claims must stay inside their evidence.

Hard stop

STOP - Holistic documentation guide complete. No implementation phase or publication started.

Source and evidence index

Paths are relative to the repository root. Click a tag in the text to reach this index; local targets are relative to this PDF location. Artifact status is prior execution inspected during synthesis, not a fresh application run. Official web sources below come from recorded repository research, not new network verification.

[S1] src/contracts.ts
InputError; parseMutation and server contract types

[S2] src/mutations.ts
processMutation; adjudicate; SQL identity and revision decisions

[S3] src/app.ts
createApp; POST admission and complete GET

[S4] src/db.ts
createPool; migrate; schema and timeout setup

[S5] src/server.ts
loopback startup and guarded process checkpoints

[S6] migrations/001_mutation_server.sql
tables, primary keys, terminal receipt constraints

[C1] client/lib/contract.ts
browser parseMutation/prepareMutation and Web Crypto

[C2] client/lib/storage.ts
LocalStore, Work/Mutation, atomic transitions and validateReceipt

[C3] client/lib/push.ts
deliver; PushController and foreground recovery

[C4] client/lib/pull.ts
validateSnapshot; pullSnapshot

[C5] client/main.tsx
App and actual action/status flow

[C6] client/conflict.tsx
ConflictPanel and explicit reviewed decisions

[D1] docs/PROJECT_CHARTER.md
purpose, bounded scope and sourced tool comparison

[D2] docs/PROTOCOL_SPEC.md
accepted behavior and chronological implementation overlays

[D3] docs/INVARIANTS_AND_FAILURE_MATRIX.md
stable invariants and F1-F10 definitions

[D4] docs/DECISION_LOG.md
selected/rejected choices and narrow clarifications

[D5] docs/ROADMAP.md
eight phases, gates and complexity plan

[D6] docs/ENGINEERING_EVIDENCE_REGISTER.md
claims, tests, artifacts and assumptions

[D7] docs/SYNCPROOF_ENGINEERING_HANDBOOK.md
existing source-grounded final-system handbook

[D8] docs/PERFORMANCE_REPORT.md
actual measurement methodology and all matrix results

[D9] docs/RELEASE_READINESS.md
public-sharing audit and unresolved owner choices

[D10] README.md
current local setup, workflow and exact supported commands

[D11] STATUS.md
current bounded project completion and stop boundary

[H1] tests/helpers.ts
generated SQL fixtures, gates and blocking observations

[H2] tests/browser/audit-harness.ts
actual isolated browser/SQL fixture and fault control

[H3] tests/audit-evidence.ts
matching-run artifact capture and destination selection

[U1] scripts/local-postgres.ps1
owned cluster startup/stop, loopback55432

[U2] scripts/bench.mjs
seeded bounded actual measurement workload

[U3] scripts/reproduce.mjs
fresh source copy/install/build/HTTP smoke

[U4] scripts/demo-clients.mjs
independent retained owner profiles and disposable check

[U5] scripts/release-audit.mjs
read-only candidate/dependency/privacy inventory

[U6] scripts/verify-release.mjs
artifact/source/statistics/document consistency checks

[U7] package.json
pinned dependencies and actual commands

[U8] vite.config.ts
loopback frontend and /api proxy

[U9] playwright.config.ts
browser worker/retry/timeout/isolation setup

[U10] package-lock.json
resolved dependency lockfile

[E1] docs/evidence/phase-7/release-check.json
latest inspected123-case release/scenario index; not a new guide-session test run

[E2] docs/evidence/phase-7/bench-raw.json
4010 actual prior measurements, summary/config/host/plans/growth

[E3] docs/evidence/phase-7/bench.txt
actual full measurement command output

[E4] docs/evidence/phase-7/F1.json
commit2/lost delivery/original replay/current3

[E5] docs/evidence/phase-7/F9.json
resolution commit6/restart/sameID/latest7

[E6] docs/evidence/phase-7/F5.json
real graceful Chromium lifecycle with actual PIDs

[E7] docs/evidence/phase-7/F6-precommit.json
real forced Node precommit termination/new PID

[E8] docs/evidence/phase-7/F6-postcommit.json
real forced Node postcommit termination/new PID

[E9] docs/evidence/phase-7/demo-run-1.json
ordered eight-step complete trace1

[E10] docs/evidence/phase-7/demo-run-2.json
ordered eight-step complete trace2

[E11] docs/evidence/phase-7/npm-audit.json
point-in-time completed registry advisory output

[E12] docs/evidence/phase-7/readiness-audit.json
candidate/private-path/license audit facts

[E13] docs/evidence/phase-7/dependency-licenses.json
installed and lock-only package metadata/notices

[E14] docs/evidence/phase-7/fresh-copy.json
independent local source-copy validation result

[E15] docs/evidence/phase-7/documentation-check.json
prior link/source check, not a substitute for this PDF review

[E16] docs/evidence/phase-6/audit-summary.json
directly inspected Phase 6 scenarios/counts/conditions

[E17] docs/evidence/phase-1/final-tests.txt
original18+27 backend execution

[E18] docs/evidence/phase-2/local-tests-final.txt
13 local/simulation cases

[E19] docs/evidence/phase-2/browser-final.txt
four original actual browser cases

[E20] docs/evidence/phase-3/browser-final.txt
15 browser cases at push milestone

[E21] docs/evidence/phase-4/snapshot-browser-final.txt
seven final snapshot browser cases

[E22] docs/evidence/phase-5/browser-strengthened.txt
31 final browser cases at resolution milestone

[X1] https://rxdb.info/replication.html
official replication documentation cited by repository research; not rechecked on the network for this PDF

[X2] https://electric.ax/docs/sync/
official read-path/Shapes overview cited by repository research

[X3] https://pouchdb.apache.org/guides/conflicts.html
official conflict/revision-tree explanation cited by repository research

[P0] docs/PHASE_REPORTS/PHASE_0.md
original Phase 0 report: scope, actual gates, commands, failures and limits

[P1] docs/PHASE_REPORTS/PHASE_1.md
original Phase 1 report: scope, actual gates, commands, failures and limits

[P2] docs/PHASE_REPORTS/PHASE_2.md
original Phase 2 report: scope, actual gates, commands, failures and limits

[P3] docs/PHASE_REPORTS/PHASE_3.md
original Phase 3 report: scope, actual gates, commands, failures and limits

[P4] docs/PHASE_REPORTS/PHASE_4.md
original Phase 4 report: scope, actual gates, commands, failures and limits

[P5] docs/PHASE_REPORTS/PHASE_5.md
original Phase 5 report: scope, actual gates, commands, failures and limits

[P6] docs/PHASE_REPORTS/PHASE_6.md
original Phase 6 report: scope, actual gates, commands, failures and limits

[P7] docs/PHASE_REPORTS/PHASE_7.md
original Phase 7 report: scope, actual gates, commands, failures and limits

[T1] tests/contracts.test.ts
actual executable test source; retained execution status comes from the referenced milestone artifacts

[T2] tests/integration.test.ts
actual executable test source; retained execution status comes from the referenced milestone artifacts

[T3] tests/process.test.ts
actual executable test source; retained execution status comes from the referenced milestone artifacts

[T4] tests/local.test.ts
actual executable test source; retained execution status comes from the referenced milestone artifacts

[T5] tests/push.test.ts
actual executable test source; retained execution status comes from the referenced milestone artifacts

[T6] tests/snapshot.test.ts
actual executable test source; retained execution status comes from the referenced milestone artifacts

[T7] tests/resolution.test.ts
actual executable test source; retained execution status comes from the referenced milestone artifacts

[T8] tests/browser/local.spec.ts
actual executable test source; retained execution status comes from the referenced milestone artifacts

[T9] tests/browser/push.spec.ts
actual executable test source; retained execution status comes from the referenced milestone artifacts

[T10] tests/browser/snapshot.spec.ts
actual executable test source; retained execution status comes from the referenced milestone artifacts

[T11] tests/browser/resolution.spec.ts
actual executable test source; retained execution status comes from the referenced milestone artifacts

[T12] tests/browser/audit.spec.ts
actual executable test source; retained execution status comes from the referenced milestone artifacts

[T13] tests/browser/demo.spec.ts
actual executable test source; retained execution status comes from the referenced milestone artifacts